

Student

I have been combining tables horizontally with JOINS — adding columns from another table. Is there a way to combine query results vertically — stacking rows from two queries together?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — that is exactly what **set operations** do. While JOINS combine tables horizontally (adding columns), set operations combine result sets **vertically** (stacking rows).

The four set operations in SQL:

- **UNION** — combines two result sets and removes duplicate rows
- **UNION ALL** — combines two result sets and keeps all rows including duplicates
- **INTERSECT** — returns only rows that appear in both result sets
- **EXCEPT** — returns rows in the first result set that do not appear in the second

🔗 ANALOGY

Think of each SELECT statement as producing a set of cards. UNION shuffles both decks together and removes duplicate cards. UNION ALL shuffles both decks and keeps every card. INTERSECT keeps only cards that appear in both decks. EXCEPT keeps only cards from the first deck that are NOT in the second deck.

🌐 REAL WORLD APPLICATION

Set operations are the backbone of cohort analysis, churn detection, and multi-source data merging. 'Which customers ordered last month AND this month?' — INTERSECT. 'Which customers ordered last month but NOT this month?' — EXCEPT. 'Combine customers from two regional databases into one deduplicated list?' — UNION.

Student

What schema are we using?

CONCEPT CARD

Same core schema plus a second dataset representing last month's data:

- `customers(customer_id, name, city, signup_date)` — current month
- `orders(order_id, customer_id, order_date, total_amount, status)` — current month
- `products(product_id, name, category, price, stock_qty)` — current month
- `last_month_orders(order_id, customer_id, order_date, total_amount, status)` — last month's orders
- `last_month_customers(customer_id, name, city, signup_date)` — last month's active customers

Think of the two datasets as data from two time periods or two regional databases with the same structure.

JOIN – Horizontal Combining

Table A

customer_id	name
(1	'Alice')
(2	'Bob')
(3	'Charlie')

Table B

customer_id	city
(1	'NY')
(2	'LA')
(3	'CHI')

JOINS add columns — wider result.

JOIN on customer_id

customer_id	name	city
(1	'Alice'	'NY')
(2	'Bob'	'LA')
(3	'Charlie'	'CHI')

SET OPERATIONS – Vertical Stacking

Result Set 1

customer_id	name
(101	'David')
(102	'Eve')
(103	'Frank')

Result Set 2

customer_id	name
(104	'Grace')
(101	'David')
(105	'Henry')

UNION / UNION ALL / INTERSECT / EXCEPT

customer_id	name
(101	'David')
(102	'Eve')
(103	'Frank')
(104	'Grace')
(101	'David')
(105	'Henry')

Set operations add rows — taller result. Same columns required.

Hard Rule: Both **SELECT** statements must return the same number of columns with compatible data types.

Set Operations — Vertical Stacking vs JOIN Horizontal Combining

Set Operations — Quick Reference Comparison					
Operation	Returns	Keeps Duplicates?	NULL Handling	Equivalent JOIN Pattern	Use Case
UNION	Rows from A plus rows from B, deduplicated	No — removes duplicates	Two NULLs treated as equal (deduplicated)	Full outer join filtering matched	Merge two custom lists,

Operation	Returns	Keeps Duplicates?	NULL Handling	Equivalent JOIN Pattern	Use Case
				rows — complex	remove duplicates
UNION ALL	All rows from A plus all rows from B	Yes — keeps all duplicates	NULLs kept as-is	No direct equivalent — UNION ALL is unique	Combine log table append data, preserve duplicates
INTERSECT	Only rows that appear in both A and B	No — removes duplicates	Two NULLs treated as equal (match)	INNER JOIN on all columns	Retained customer — ordered both months
EXCEPT	Rows in A that do not appear in B	No — removes duplicates	Two NULLs treated as equal (excluded)	LEFT JOIN ... WHERE b.col IS NULL (anti-join)	Churned customer — ordered last month, not this month
EXCEPT (Oracle: MINUS)	Same as EXCEPT	No	Same as EXCEPT	Same as EXCEPT	Oracle syntax only — same semantics

sql – Set Operation Syntax Templates – All Four Types

```
-- UNION: combine and deduplicate
SELECT customer_id, name, city FROM customers
UNION
SELECT customer_id, name, city FROM last_month_customers;

-- UNION ALL: combine and keep duplicates
SELECT customer_id, name, city FROM customers
UNION ALL
```

```

SELECT customer_id, name, city FROM last_month_customers;

-- INTERSECT: rows in both
SELECT customer_id FROM orders
INTERSECT
SELECT customer_id FROM last_month_orders;

-- EXCEPT: rows in first but not second
SELECT customer_id FROM last_month_orders
EXCEPT
SELECT customer_id FROM orders;

```

THE ONE HARD RULE — COLUMN COUNT AND DATA TYPE COMPATIBILITY

ChatGPT • AI Assistant

A three-row diagram illustrating the column-matching rule. Row 1 (green background, label 'VALID'): two SELECT boxes side by side, each with three columns labeled 'INT | VARCHAR | VARCHAR' in matching color bands. An equals sign between them reads 'Same count, compatible types — OK.' Row 2 (red background, label 'INVALID — Different Column Count'): left SELECT has three columns (INT | VARCHAR | VARCHAR), right SELECT has only two columns (INT | VARCHAR). A red X between them reads 'ERROR: column count mismatch.' Row 3 (orange background, label 'CAREFUL — Type Mismatch'): left SELECT columns labeled 'INT | VARCHAR', right SELECT columns labeled 'VARCHAR | INT' (swapped). An orange warning icon between them reads 'Silent wrong result or cast error — SQL matches by position, not by name.' A rule box at the bottom reads: 'Column names come from the FIRST SELECT. ORDER BY goes at the very end. Pad missing columns with NULL.' Clean grid layout, color-coded column type bands.

The Hard Rule — Column Position and Type Matching

sql — The Hard Rule — Valid and Invalid Set Operation Examples

```

-- VALID: same column count and compatible types
SELECT customer_id, name, city FROM customers
UNION
SELECT customer_id, name, city FROM last_month_customers;

-- INVALID: different column count
-- SELECT customer_id, name, city FROM customers
-- UNION
-- SELECT customer_id, name FROM last_month_customers; -- ERROR

-- Padding with NULL for missing columns
SELECT customer_id, name, city FROM customers
UNION ALL

```

```
SELECT customer_id, name, NULL AS city FROM archived_customers;

-- Column names come from the FIRST SELECT
SELECT customer_id AS id, name AS customer_name FROM customers
UNION
SELECT customer_id, name FROM last_month_customers;
-- Result columns are: id, customer_name

-- ORDER BY at the end only
SELECT customer_id, name FROM customers
UNION
SELECT customer_id, name FROM last_month_customers
ORDER BY name ASC;  -- applies to entire combined result

-- WRONG: ORDER BY inside a SELECT
-- SELECT customer_id, name FROM customers ORDER BY name -- ERROR
-- UNION
-- SELECT customer_id, name FROM last_month_customers;
```

Student

Before we dive into each operation — you said there is one hard rule. What is it?

ChatGPT • AI Assistant

CONCEPT CARD

The **one hard rule** for all set operations:

Both SELECT statements must return the same number of columns, and the corresponding columns must have compatible data types.

The column names in the final result are taken from the **first** SELECT statement. The second SELECT's column names are ignored.

EXAMPLE CARD**Example 1 — Valid: Same column count and compatible types**

sql — Code Example

```
SELECT customer_id, name, city
FROM customers
UNION
```

```
SELECT customer_id, name, city
FROM last_month_customers;
-- OK: 3 columns, same types (INT, VARCHAR, VARCHAR)
```

Example 2 — Invalid: Different column counts

sql – Code Example

```
SELECT customer_id, name, city
FROM customers
UNION
SELECT customer_id, name
FROM last_month_customers;
-- ERROR: each UNION query must have the same number of columns
```

Example 3 — Invalid: Incompatible data types

sql – Code Example

```
SELECT customer_id, name
FROM customers
UNION
SELECT name, customer_id -- swapped: VARCHAR UNION INT for first
column
FROM last_month_customers;
-- ERROR or implicit cast failure – types do not match positionally
```

⚠ WARNING

SQL matches columns **by position**, not by name. If you accidentally swap the column order in one of the SELECT statements, the database may silently cast types or throw an error — the data will be wrong either way.

Example 4 — Using NULL as a column placeholder

sql – Code Example

```
-- Table A has 3 columns, Table B has only 2 – pad with NULL
SELECT customer_id, name, city
FROM customers
UNION ALL
SELECT customer_id, name, NULL AS city -- placeholder for missing
column
FROM archived_customers;
```

This is a common real-world pattern when combining tables that have slightly different schemas — pad the missing columns with NULL.

Example 5 — Column names come from the first SELECT

sql — Code Example

```
SELECT customer_id AS id, name AS customer_name
FROM customers
UNION
SELECT customer_id, name    -- these aliases are IGNORED
FROM last_month_customers;
-- Result columns are named: id, customer_name (from first SELECT)
```

Student

Can I use ORDER BY with set operations?

ChatGPT • AI Assistant

Yes — but ORDER BY applies to the **entire combined result**, not to individual SELECT statements. It goes at the very end, after the last SELECT:

sql — Code Example

```
SELECT customer_id, name FROM customers
UNION
SELECT customer_id, name FROM last_month_customers
ORDER BY name ASC;    -- sorts the combined result
```

❗ COMMON PITFALL / MISTAKE

✗ **ORDER BY inside one of the SELECT statements — not allowed:**

sql — Code Example

```
SELECT customer_id, name FROM customers ORDER BY name -- ERROR
UNION
SELECT customer_id, name FROM last_month_customers;
```

✓ **Correct — ORDER BY at the end only:**

sql — Code Example

```
SELECT customer_id, name FROM customers
UNION
SELECT customer_id, name FROM last_month_customers
ORDER BY name;
```

MEMORY TRICK CARD

Remember the rule as: **SAME WIDTH, COMPATIBLE TYPES, ONE ORDER BY AT THE END.**

- Same width = same number of columns.
- Compatible types = positionally matching data types.
- One ORDER BY = at the end of the entire set operation, not inside any SELECT.

 INTERVIEW CARD

Frequency: High

Question: What are the requirements for using set operations like UNION or INTERSECT?

Expected Answer: Both SELECT statements must return the same number of columns, and the corresponding columns (matched by position, not name) must have compatible data types. Column names in the result come from the first SELECT statement. ORDER BY can only appear once at the very end of the combined query — it applies to the entire result, not to individual SELECT statements. If a table is missing a column, pad it with NULL AS column_name.

Follow-up: How do column names work in a UNION result? (Answer: The column names are taken from the first SELECT statement. Aliases in the second or subsequent SELECT statements are completely ignored.)

 PRACTICE QUIZ

1. What error do you get if both SELECT statements in a UNION return a different number of columns?
2. Which SELECT statement's column names appear in the final UNION result?
3. How do you handle a UNION between two tables where one table is missing a column?
4. Where must ORDER BY appear in a set operation query?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. 'Each UNION query must have the same number of columns' — a syntax or runtime error.
2. The first SELECT statement's column names (or aliases) are used in the result.
3. Pad the missing column with NULL AS column_name in the SELECT with fewer columns.
4. At the very end of the entire set operation — after the last SELECT statement. One ORDER BY for the whole combined result.

CHECKPOINT QUESTION

This query has two mistakes. Find them and write the corrected version:

sql – Code Example

```
SELECT customer_id, name, city FROM customers ORDER BY name
UNION
SELECT customer_id, city FROM last_month_customers;
```

UNION AND UNION ALL — COMBINING AND DEDUPLICATION

ChatGPT • AI Assistant

A three-panel diagram. Top-left panel (label 'Result Set A — 4 rows'): a table with columns customer_id and name showing rows: 101/Amelia, 102/Rohan, 103/Sara, 104/Priya. Top-right panel (label 'Result Set B — 4 rows'): a table showing rows: 102/Rohan, 103/Sara, 105/Dave, 106/Meena. Note that rows 102/Rohan and 103/Sara appear in both. Bottom-left panel (label 'UNION result — 6 rows'): combined table showing 6 unique rows: 101/Amelia, 102/Rohan, 103/Sara, 104/Priya, 105/Dave, 106/Meena. Rows 102 and 103 from Set B are struck through in red with label 'duplicates removed.' Bottom-right panel (label 'UNION ALL result — 8 rows'): combined table showing all 8 rows including the duplicate 102/Rohan and 103/Sara rows highlighted in orange with label 'duplicates kept.' Performance note box reads: 'UNION ALL is faster — skips sort/hash deduplication step. Prefer UNION ALL unless deduplication is required.' Clean four-panel grid, consistent colors.

UNION vs UNION ALL — Deduplication Behavior Visualized

sql – UNION and UNION ALL – Deduplication, Performance, and Aggregation Pattern

```
-- UNION: deduplicated combined customer list
SELECT customer_id, name FROM customers
UNION
SELECT customer_id, name FROM last_month_customers
ORDER BY name;

-- UNION ALL: all rows kept (faster)
```

```
SELECT order_id, customer_id, order_date, total_amount, status
FROM orders
UNION ALL
SELECT order_id, customer_id, order_date, total_amount, status
FROM last_month_orders
ORDER BY order_date DESC;

-- UNION ALL in derived table then aggregate
SELECT SUM(total_amount) AS combined_revenue,
       COUNT(*)          AS combined_order_count
FROM (
    SELECT total_amount FROM orders
    UNION ALL
    SELECT total_amount FROM last_month_orders
) all_orders;

-- NULL deduplication in UNION
SELECT NULL AS city UNION      SELECT NULL AS city; -- 1 row (NULL deduplicated)
SELECT NULL AS city UNION ALL SELECT NULL AS city; -- 2 rows (both kept)

-- Multi-source monthly revenue (12 tables – use UNION ALL)
SELECT order_month, SUM(revenue) AS total_revenue
FROM (
    SELECT 'Jan' AS order_month, total_amount AS revenue FROM
orders_jan
    UNION ALL
    SELECT 'Feb',                total_amount                FROM
orders_feb
    UNION ALL
    SELECT 'Mar',                total_amount                FROM
orders_mar
) monthly
GROUP BY order_month
ORDER BY order_month;
```

Student

Let us go through each operation in detail. Start with UNION.

CONCEPT CARD

UNION combines two result sets and **removes duplicate rows**. A row is considered a duplicate if every column value matches exactly — including NULLs (two NULLs are treated as equal for deduplication purposes, unlike in WHERE clauses).

UNION ALL does the same combination but **keeps all rows including duplicates** — and is significantly faster because it skips the deduplication step.

🔗 ANALOGY

UNION is like merging two email subscriber lists and removing people who appear on both. UNION ALL is like printing both lists end-to-end without checking for duplicates — faster, but you may have the same person twice.

EXAMPLE CARD**Example 1 — UNION: Combine and deduplicate customer lists from two regions**

sql — Code Example

```
-- North region customers
SELECT customer_id, name, city
FROM customers
WHERE city IN ('Delhi', 'Chandigarh', 'Jaipur')

UNION

-- South region customers (same table, different filter – simulating
two sources)
SELECT customer_id, name, city
FROM customers
WHERE city IN ('Hyderabad', 'Bangalore', 'Chennai')

ORDER BY city, name;
```

Because these are different cities, there will be no duplicates here. UNION still performs the deduplication scan.

Example 2 — UNION with overlapping data (deduplication in action)

sql — Code Example

```
-- Active customers this month
SELECT customer_id, name
FROM customers
```

UNION

```
-- Active customers last month (same customers may appear in both)
SELECT customer_id, name
FROM last_month_customers

ORDER BY name;
```

Output: Every unique customer who was active in either month — customers who were active in both months appear **only once**.

Example 3 — UNION ALL: Combine order logs from two months (keep all rows)

sql – Code Example

```
SELECT order_id, customer_id, order_date, total_amount, status
FROM orders

UNION ALL

SELECT order_id, customer_id, order_date, total_amount, status
FROM last_month_orders

ORDER BY order_date DESC;
```

Output: Every order from both months — if the same order_id exists in both tables, it appears twice. Use UNION ALL here because orders from different months are distinct and we want all of them.

Example 4 — UNION ALL then aggregate: Total revenue across both months

sql – Code Example

```
SELECT SUM(total_amount) AS combined_revenue,
       COUNT(*)           AS combined_order_count
FROM (
    SELECT total_amount FROM orders
    UNION ALL
    SELECT total_amount FROM last_month_orders
) all_orders;
```

This pattern — UNION ALL inside a derived table, then aggregate outside — is a common production pattern for combining multiple data sources before computing totals.

When should I always choose UNION ALL over UNION?

ChatGPT • AI Assistant

Choose UNION ALL whenever:

1. You **know** there are no duplicates — e.g., combining orders from different months using different date ranges.
2. **Duplicates are acceptable** — e.g., appending log rows where duplicates are meaningful.
3. **Performance matters** — UNION ALL skips the deduplication step (which internally does a sort or hash) and is always faster.

Choose UNION only when you genuinely need to remove duplicates — e.g., combining customer lists from two sources where the same customer may appear in both.

⚠ **WARNING**

NULL handling in UNION deduplication is different from WHERE clauses:

- In WHERE: `NULL = NULL` evaluates to NULL (not TRUE) — NULLs do not match.
- In UNION deduplication: two rows with NULL in the same column **are considered equal** and one is removed.

sql — Code Example

```
-- UNION treats two NULLs as equal for deduplication:
SELECT NULL AS city
UNION
SELECT NULL AS city;
-- Result: one row with NULL (duplicates removed)

SELECT NULL AS city
UNION ALL
SELECT NULL AS city;
-- Result: two rows with NULL (kept)
```

EXAM TIP CARD

In interviews, when asked 'When do you prefer UNION ALL over UNION?' — the answer is: **always prefer UNION ALL unless you specifically need deduplication.**

UNION ALL is faster because it avoids the sort/hash deduplication step. Many developers default to UNION out of habit — using UNION ALL when appropriate shows you understand the performance difference.

INTERVIEW CARD

Frequency: Very High

Question: What is the difference between UNION and UNION ALL? When would you choose each?

Expected Answer: UNION combines two result sets and removes duplicate rows using an internal sort or hash operation — slower. UNION ALL combines two result sets and keeps all rows including duplicates — faster because no deduplication is performed. Choose UNION ALL when you know there are no duplicates (different time periods, different keys), when duplicates are acceptable, or when performance matters. Choose UNION only when you genuinely need deduplication across the combined result. Note that UNION treats two NULLs as equal for deduplication, unlike WHERE clauses.

Follow-up: You have orders from 12 monthly tables and want to compute total annual revenue. Which do you use and why? (Answer: UNION ALL — orders from different months cannot be duplicates since they have different order dates and IDs. UNION would waste time scanning for non-existent duplicates.)

PRACTICE QUIZ

1. Which is faster: UNION or UNION ALL? Why?
2. How does UNION treat two NULL values in the same column — are they considered duplicates?
3. You are combining order logs from 12 monthly tables to compute total revenue. Which operation do you use?
4. Predict: how many rows does this return?

sql — Code Example

```
SELECT 1 AS n UNION SELECT 1 AS n UNION SELECT 2 AS n;
```

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. UNION ALL is faster — it skips the deduplication step (no internal sort or hash required).
2. Yes — in UNION deduplication, two NULLs in the same column are treated as equal and one is removed. This is different from WHERE where NULL = NULL evaluates to NULL (not TRUE).
3. UNION ALL — orders from different months have different order dates and IDs, so there are no real duplicates. UNION would scan for duplicates unnecessarily.
4. Two rows: (1) and (2) — the two SELECT 1 rows are duplicates and UNION removes one.

🔍 CHECKPOINT QUESTION

You have two tables: `app_users(user_id, email, signup_date)` from your web app and `mobile_users(user_id, email, signup_date)` from your mobile app. The same user may have accounts in both. Write a query that produces a deduplicated list of all users ever (use email as the unique identifier) and a separate query that produces the full list keeping duplicates. Explain when you would use each.

INTERSECT — ROWS THAT APPEAR IN BOTH RESULT SETS

ChatGPT • AI Assistant

A Venn diagram with two large overlapping circles. Left circle (blue, label 'This Month Orders — 312 customers'): filled light blue. Right circle (orange, label 'Last Month Orders — 298 customers'): filled light orange. The overlapping center section is filled dark green with label 'INTERSECT — 187 customers ordered BOTH months = Retained Customers.' The left-only section (blue) is labeled 'New customers this month — not in INTERSECT.' The right-only section (orange) is labeled 'Churned customers — ordered last month but not this month.' Below the Venn, a SQL snippet reads: 'SELECT customer_id FROM orders INTERSECT SELECT customer_id FROM last_month_orders.' A business metric callout box reads: 'Retention Rate = INTERSECT count / Last Month count = 187 / 298 = 62.8%.' Clean flat Venn diagram design, consistent color coding.

INTERSECT — Retained Customers Cohort Analysis

sql — INTERSECT — Retained Customers, JOIN Rewrite, and Cohort Analysis

```
-- Basic INTERSECT: retained customers (ordered both months)
SELECT customer_id FROM orders
INTERSECT
SELECT customer_id FROM last_month_orders
ORDER BY customer_id;

-- Equivalent JOIN rewrite (use for MySQL)
SELECT DISTINCT o.customer_id
FROM orders o
```

```
INNER JOIN last_month_orders lmo ON o.customer_id = lmo.customer_id;

-- Retained customers with details (INTERSECT inside IN)
SELECT c.customer_id, c.name, c.city
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM orders
    INTERSECT
    SELECT customer_id FROM last_month_orders
)
ORDER BY c.city, c.name;

-- Products ordered in both months
SELECT product_id
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
INTERSECT
SELECT product_id
FROM order_items oi2
JOIN last_month_orders lmo ON oi2.order_id = lmo.order_id
ORDER BY product_id;

-- INTERSECT with date filter (Jan 2024 AND Feb 2024 customers)
SELECT customer_id
FROM orders
WHERE order_date ≥ '2024-01-01' AND order_date < '2024-02-01'
INTERSECT
SELECT customer_id
FROM orders
WHERE order_date ≥ '2024-02-01' AND order_date < '2024-03-01';
```

Student

Now INTERSECT. I want to find customers who ordered both last month AND this month — retained customers. How does that work?

ChatGPT • AI Assistant

CONCEPT CARD

INTERSECT returns only the rows that appear in **both** result sets — like a mathematical set intersection. It removes duplicates from the result by default.

sql – Code Example

```
SELECT customer_id FROM orders
INTERSECT
SELECT customer_id FROM last_month_orders;
```

This returns customer IDs that appear in both the current month's orders AND last month's orders — i.e., customers who ordered in both months. These are your **retained customers**.

ANALOGY

If UNION is the OR of two sets, INTERSECT is the AND. Only members of both clubs get a result.

EXAMPLE CARD

Example 1 — Basic INTERSECT: Retained customers (ordered both months)

sql – Code Example

```
SELECT customer_id
FROM orders

INTERSECT

SELECT customer_id
FROM last_month_orders

ORDER BY customer_id;
```

Output: List of customer IDs who placed at least one order this month AND at least one order last month.

Example 2 — INTERSECT with more columns: Products ordered in both months

sql – Code Example

```
SELECT product_id
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id

INTERSECT

SELECT product_id
FROM order_items oi
```

```
JOIN last_month_orders lmo ON oi.order_id = lmo.order_id

ORDER BY product_id;
```

Example 3 — INTERSECT rewritten as a JOIN (equivalent)

sql – Code Example

```
-- INTERSECT version:
SELECT customer_id FROM orders
INTERSECT
SELECT customer_id FROM last_month_orders;

-- Equivalent JOIN rewrite:
SELECT DISTINCT o.customer_id
FROM orders o
INNER JOIN last_month_orders lmo ON o.customer_id = lmo.customer_id
ORDER BY o.customer_id;
```

Both produce the same result. The JOIN rewrite uses DISTINCT to replicate the deduplication behavior of INTERSECT.

Example 4 — Practical cohort query: Retained customers with their details

sql – Code Example

```
SELECT c.customer_id,
       c.name,
       c.city
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM orders
    INTERSECT
    SELECT customer_id FROM last_month_orders
)
ORDER BY c.city, c.name;
```

The INTERSECT is used inside an IN subquery to get the retained customer IDs, then the outer query joins back to get the customer details.

Student

Does INTERSECT check all columns or just specific ones?

INTERSECT compares rows across **all columns** in the result set. Two rows are considered matching only if every single column value matches.

sql – Code Example

```
-- This checks both customer_id AND city match in both result sets:
SELECT customer_id, city FROM customers
INTERSECT
SELECT customer_id, city FROM last_month_customers;
-- Returns only rows where BOTH customer_id and city are the same
```

If you only want to match on one column (like customer_id), SELECT only that column in both queries.

⚠ WARNING

Not all databases support INTERSECT. PostgreSQL, SQL Server, Oracle, and SQLite support it. **MySQL does NOT support INTERSECT** (before MySQL 8.0.31) — use the JOIN rewrite instead.

🧠 INTERVIEW CARD

Frequency: Medium

Question: What does INTERSECT return? How would you rewrite it as a JOIN?

Expected Answer: INTERSECT returns rows that appear in both result sets, deduplicating the result. It compares across all selected columns. The JOIN equivalent is: INNER JOIN on all selected columns with DISTINCT applied. For example, `SELECT customer_id FROM A INTERSECT SELECT customer_id FROM B` equals `SELECT DISTINCT a.customer_id FROM A a INNER JOIN B b ON a.customer_id = b.customer_id`. Use INTERSECT for clarity when both semantics and database support allow; use the JOIN rewrite for MySQL compatibility.

Follow-up: Does INTERSECT ALL exist? (Answer: In PostgreSQL yes — INTERSECT ALL keeps duplicate matching rows. Rarely used in practice.)

🧐 PRACTICE QUIZ

1. What does INTERSECT return if no rows are common between both result sets?
2. INTERSECT checks column matches by position or by name?

3. Rewrite as a JOIN: `SELECT product_id FROM order_items INTERSECT SELECT product_id FROM last_month_order_items`
4. Does MySQL (pre-8.0.31) support INTERSECT?

▼ **Reveal Correct Answer**

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 **ANSWER KEY**

1. An empty result set — zero rows.
2. By position — all set operations match columns positionally.
3. `SELECT DISTINCT oi.product_id FROM order_items oi INNER JOIN last_month_order_items lmoi ON oi.product_id = lmoi.product_id;`
4. No — use the INNER JOIN + DISTINCT rewrite for MySQL compatibility.

❓ **CHECKPOINT QUESTION**

You are a data analyst at an e-commerce company. The product team asks: 'Which customers were active (placed at least one non-cancelled order) in both January 2024 and February 2024?' Write the INTERSECT query and then write the JOIN equivalent. Which would you use in a MySQL environment?

EXCEPT — ROWS IN THE FIRST SET NOT IN THE SECOND

ChatGPT • AI Assistant

A two-row diagram showing order sensitivity of EXCEPT. Top row (label 'A EXCEPT B — Churned Customers'): two circles side by side, left circle labeled 'A = Last Month Orders (298 customers)' filled light orange, right circle labeled 'B = This Month Orders (312 customers)' filled light blue. The left-only section of circle A is filled dark red with label 'EXCEPT result: 111 customers — ordered last month, NOT this month = CHURNED.' Bottom row (label 'B EXCEPT A — New Customers'): same two circles but the right-only section of circle B is filled dark green with label 'EXCEPT result: 125 customers — ordered this month, NOT last month = NEW CUSTOMERS.' A bold warning box between the two rows reads: 'ORDER MATTERS — swapping the SELECT statements gives a completely different business answer!' A third box shows the Oracle equivalent: 'Oracle uses MINUS — same semantics.' Clean diagram, consistent color coding.

EXCEPT — Churn Detection and Order Matters

sql — EXCEPT — Churned Customers, New Customers, and Anti-Join Rewrite

```
-- EXCEPT: churned customers (ordered last month, not this month)
SELECT customer_id FROM last_month_orders
EXCEPT
SELECT customer_id FROM orders
```

```
ORDER BY customer_id;

-- Reversed EXCEPT: new customers this month
SELECT customer_id FROM orders
EXCEPT
SELECT customer_id FROM last_month_orders
ORDER BY customer_id;

-- Anti-join rewrite (MySQL compatible)
SELECT DISTINCT lmo.customer_id
FROM last_month_orders lmo
LEFT JOIN orders o ON lmo.customer_id = o.customer_id
WHERE o.customer_id IS NULL;

-- Churned customers with details
SELECT c.customer_id, c.name, c.city, c.signup_date
FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM last_month_orders
    EXCEPT
    SELECT customer_id FROM orders
)
ORDER BY c.city, c.name;

-- Oracle equivalent (MINUS)
-- SELECT customer_id FROM last_month_orders
-- MINUS
-- SELECT customer_id FROM orders;

-- Products active last month but not this month
SELECT DISTINCT product_id
FROM order_items oi JOIN last_month_orders lmo ON oi.order_id =
lmo.order_id
EXCEPT
SELECT DISTINCT product_id
FROM order_items oi2 JOIN orders o ON oi2.order_id = o.order_id
ORDER BY product_id;
```

Student

Now EXCEPT. I want to find churned customers — people who ordered last month but did NOT order this month. That sounds like EXCEPT.

CONCEPT CARD

EXCEPT returns rows from the **first** result set that do NOT appear in the **second** result set. It removes duplicates from the result. The order of the two SELECT statements matters — swapping them gives a completely different answer.

sql – Code Example

```
-- Customers who ordered last month but NOT this month (churned)
SELECT customer_id FROM last_month_orders
EXCEPT
SELECT customer_id FROM orders;
```

🔗 ANALOGY

EXCEPT is like subtracting one set from another. Start with last month's guest list, cross off everyone who also appears on this month's guest list — whoever is left is your churned set.

⚠️ WARNING

Order matters in EXCEPT. $A \text{ EXCEPT } B \neq B \text{ EXCEPT } A$.

- $A \text{ EXCEPT } B$ = rows in A not in B (churned from A's perspective).
- $B \text{ EXCEPT } A$ = rows in B not in A (new in A, missing from B).

EXAMPLE CARD

Example 1 — Basic EXCEPT: Churned customers

sql – Code Example

```
-- Customers who ordered last month but NOT this month
SELECT customer_id
FROM last_month_orders

EXCEPT

SELECT customer_id
FROM orders

ORDER BY customer_id;
```

Output: Customer IDs of customers who were active last month but placed no order this month — your churned customers.

Example 2 — Reversed EXCEPT: New customers (ordered this month but not last)

sql – Code Example

```
-- New customers this month (no order history last month)
SELECT customer_id
FROM orders

EXCEPT

SELECT customer_id
FROM last_month_orders

ORDER BY customer_id;
```

Swapping the two SELECT statements gives you new customers instead of churned customers — completely different business meaning.

Example 3 — EXCEPT rewritten as anti-join (equivalent)

sql – Code Example

```
-- EXCEPT version:
SELECT customer_id FROM last_month_orders
EXCEPT
SELECT customer_id FROM orders;

-- Equivalent anti-join rewrite:
SELECT DISTINCT lmo.customer_id
FROM last_month_orders lmo
LEFT JOIN orders o ON lmo.customer_id = o.customer_id
WHERE o.customer_id IS NULL;
```

Both return the same result. The anti-join is the preferred approach in MySQL (which may not support EXCEPT in older versions).

Example 4 — Churn analysis with customer details

sql – Code Example

```
SELECT c.customer_id,
       c.name,
       c.city,
       c.signup_date
FROM customers c
```

```
WHERE c.customer_id IN (  
    SELECT customer_id FROM last_month_orders  
    EXCEPT  
    SELECT customer_id FROM orders  
)  
ORDER BY c.city, c.name;
```

Example 5 — Real business use: Products discontinued this month

sql – Code Example

```
-- Products that were ordered last month but have NOT been ordered  
this month  
SELECT DISTINCT product_id  
FROM order_items oi  
JOIN last_month_orders lmo ON oi.order_id = lmo.order_id  
  
EXCEPT  
  
SELECT DISTINCT product_id  
FROM order_items oi2  
JOIN orders o ON oi2.order_id = o.order_id  
  
ORDER BY product_id;
```

Student

What is the Oracle equivalent of EXCEPT?

ChatGPT • AI Assistant

Oracle uses **MINUS** instead of **EXCEPT** — they are semantically identical:

sql – Code Example

```
-- Oracle syntax  
SELECT customer_id FROM last_month_orders  
MINUS  
SELECT customer_id FROM orders;  
  
-- Standard SQL / PostgreSQL / SQL Server  
SELECT customer_id FROM last_month_orders
```

```
EXCEPT
SELECT customer_id FROM orders;
```

Learn both keywords — you will see MINUS in Oracle environments and EXCEPT everywhere else.

INTERVIEW CARD

Frequency: Medium-High

Question: What does EXCEPT return? How is it different from INTERSECT?

Expected Answer: EXCEPT returns rows from the first result set that do not appear in the second result set — like set subtraction. INTERSECT returns only rows that appear in both result sets — like set intersection. In EXCEPT, the order of SELECT statements matters: A EXCEPT B gives different results than B EXCEPT A. Both operations deduplicate their output. The EXCEPT equivalent using joins is the anti-join pattern: LEFT JOIN with WHERE right.key IS NULL. Oracle calls this operation MINUS instead of EXCEPT.

Follow-up: What is the MySQL equivalent of EXCEPT? (Answer: MySQL (pre-8.0.31) does not support EXCEPT natively — use the anti-join: LEFT JOIN ... WHERE right.key IS NULL, or a NOT IN / NOT EXISTS subquery.)

PRACTICE QUIZ

1. Does `A EXCEPT B` return the same result as `B EXCEPT A` ?
2. What is the Oracle keyword equivalent to EXCEPT?
3. Write the anti-join equivalent of: `SELECT customer_id FROM last_month_orders EXCEPT SELECT customer_id FROM orders`
4. You want to find products that appear in this month's orders but NOT in last month's orders. Write the EXCEPT query.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

ANSWER KEY

1. No — order matters completely. A EXCEPT B returns rows in A not in B. B EXCEPT A returns rows in B not in A.
2. MINUS — e.g., `SELECT ... FROM A MINUS SELECT ... FROM B`
3. `SELECT DISTINCT lmo.customer_id FROM last_month_orders lmo LEFT JOIN orders o ON lmo.customer_id = o.customer_id WHERE o.customer_id IS NULL`

```
NULL;  
4. SELECT DISTINCT product_id FROM order_items oi JOIN orders o ON  
   oi.order_id = o.order_id EXCEPT SELECT DISTINCT product_id FROM  
   order_items oi2 JOIN last_month_orders lmo ON oi2.order_id =  
   lmo.order_id;
```

🔗 CHECKPOINT QUESTION

Your manager asks for a monthly churn report: the count of customers who ordered in March 2024 but did NOT order in April 2024, broken down by city. Write the full query using EXCEPT inside a subquery, then join to customers to get the city. Include a count per city sorted by churn count descending.

COMBINING SET OPERATIONS AND 10 PRACTICE PROBLEMS

ChatGPT • AI Assistant

A three-circle Venn-style diagram showing cohort analysis. Center-left: circle labeled 'Last Month Orders — 298 unique customers' filled light orange. Center-right: circle labeled 'This Month Orders — 312 unique customers' filled light blue. Three annotated regions with result counts and SQL labels. Region 1 (orange only, left non-overlap): labeled 'CHURNED — 111 customers. Last Month EXCEPT This Month.' An orange box shows the SQL snippet. Region 2 (green overlap): labeled 'RETAINED — 187 customers. This Month INTERSECT Last Month.' A green box shows the SQL snippet. Region 3 (blue only, right non-overlap): labeled 'NEW — 125 customers. This Month EXCEPT Last Month.' A blue box shows the SQL snippet. Below the diagram, a summary box shows: 'Total covered: 111 + 187 + 125 = 423 unique customers across both months. Cohort report combines all three with UNION ALL.' Clean Venn design, monospace SQL snippets, consistent color coding.

Complete Cohort Analysis Using All Three Set Operations

```
sql - 10 Practice Problems - Complete Solutions  
  
-- Problem 1: UNION deduplication  
SELECT customer_id, name, city FROM customers  
UNION  
SELECT customer_id, name, city FROM last_month_customers  
ORDER BY name;  
  
-- Problem 2: UNION ALL preserve all rows  
SELECT order_id, customer_id, order_date, total_amount, status FROM  
orders  
UNION ALL  
SELECT order_id, customer_id, order_date, total_amount, status FROM  
last_month_orders  
ORDER BY order_date DESC;  
  
-- Problem 3: INTERSECT retained customers
```

```
SELECT customer_id FROM orders
INTERSECT
SELECT customer_id FROM last_month_orders
ORDER BY customer_id;

-- Problem 4: EXCEPT churned customers
SELECT customer_id FROM last_month_orders
EXCEPT
SELECT customer_id FROM orders
ORDER BY customer_id;

-- Problem 5: UNION ALL combined revenue
SELECT SUM(total_amount) AS combined_revenue, COUNT(*) AS
combined_orders
FROM (SELECT total_amount FROM orders UNION ALL SELECT total_amount
FROM last_month_orders) a;

-- Problem 6: INTERSECT with customer details
SELECT c.customer_id, c.name, c.city FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM orders
    INTERSECT SELECT customer_id FROM last_month_orders
) ORDER BY c.city, c.name;

-- Problem 7: EXCEPT churned with details
SELECT c.customer_id, c.name, c.city, c.signup_date FROM customers c
WHERE c.customer_id IN (
    SELECT customer_id FROM last_month_orders
    EXCEPT SELECT customer_id FROM orders
) ORDER BY c.signup_date;

-- Problem 8: EXCEPT rewritten as anti-join
SELECT DISTINCT c.customer_id, c.name, c.city
FROM customers c
JOIN last_month_orders lmo ON c.customer_id = lmo.customer_id
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.customer_id IS NULL ORDER BY c.name;

-- Problem 9: Full cohort report
SELECT 'New' AS cohort, COUNT(*) AS customer_count
FROM (SELECT customer_id FROM orders EXCEPT SELECT customer_id FROM
last_month_orders) n
UNION ALL
SELECT 'Retained', COUNT(*)
FROM (SELECT customer_id FROM orders INTERSECT SELECT customer_id FROM
last_month_orders) r
```

```
UNION ALL
SELECT 'Churned', COUNT(*)
FROM (SELECT customer_id FROM last_month_orders EXCEPT SELECT
customer_id FROM orders) c
ORDER BY cohort;

-- Problem 10: Monthly revenue breakdown Jan-Mar 2024
SELECT order_month, COUNT(*) AS order_count, SUM(total_amount) AS
monthly_revenue
FROM (
    SELECT 'Jan 2024' AS order_month, total_amount FROM orders
    WHERE order_date ≥ '2024-01-01' AND order_date < '2024-02-01'
    UNION ALL
    SELECT 'Feb 2024', total_amount FROM orders
    WHERE order_date ≥ '2024-02-01' AND order_date < '2024-03-01'
    UNION ALL
    SELECT 'Mar 2024', total_amount FROM orders
    WHERE order_date ≥ '2024-03-01' AND order_date < '2024-04-01'
) monthly_data
GROUP BY order_month
ORDER BY order_month;
```

Student

Can I chain multiple set operations together? Like UNION then EXCEPT?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — you can chain multiple set operations in sequence. They are evaluated **left to right** by default. Use parentheses to control the order explicitly, just like arithmetic.

sql — Code Example

```
-- Left to right: (A UNION B) EXCEPT C
SELECT customer_id FROM customers
UNION
SELECT customer_id FROM last_month_customers
EXCEPT
SELECT customer_id FROM orders
WHERE status = 'cancelled';
```

```
-- Explicit order with parentheses:  
(SELECT customer_id FROM customers  
  UNION  
  SELECT customer_id FROM last_month_customers)  
EXCEPT  
(SELECT customer_id FROM orders WHERE status = 'cancelled');
```

EXAMPLE CARD**Example 1 — Chaining UNION ALL and GROUP BY: Full monthly revenue report**

sql – Code Example

```
SELECT order_month,  
       COUNT(*)      AS order_count,  
       SUM(total_amount) AS revenue  
FROM (  
  SELECT 'This Month' AS order_month, total_amount FROM orders  
  UNION ALL  
  SELECT 'Last Month' AS order_month, total_amount FROM  
last_month_orders  
) combined  
GROUP BY order_month  
ORDER BY order_month;
```

Example 2 — Three-way UNION ALL: Combine three months of data

sql – Code Example

```
SELECT 'January' AS month, SUM(total_amount) AS revenue FROM  
orders_jan  
UNION ALL  
SELECT 'February' AS month, SUM(total_amount) AS revenue FROM  
orders_feb  
UNION ALL  
SELECT 'March' AS month, SUM(total_amount) AS revenue FROM  
orders_mar  
ORDER BY month;
```

Example 3 — Complete cohort analysis: New, Retained, and Churned counts

sql – Code Example

```
-- New customers (this month, not last month)  
SELECT 'New' AS cohort, COUNT(*) AS customer_count
```

```
FROM (  
    SELECT customer_id FROM orders  
    EXCEPT  
    SELECT customer_id FROM last_month_orders  
) new_customers  
  
UNION ALL  
  
-- Retained customers (both months)  
SELECT 'Retained' AS cohort, COUNT(*) AS customer_count  
FROM (  
    SELECT customer_id FROM orders  
    INTERSECT  
    SELECT customer_id FROM last_month_orders  
) retained_customers  
  
UNION ALL  
  
-- Churned customers (last month, not this month)  
SELECT 'Churned' AS cohort, COUNT(*) AS customer_count  
FROM (  
    SELECT customer_id FROM last_month_orders  
    EXCEPT  
    SELECT customer_id FROM orders  
) churned_customers  
  
ORDER BY cohort;
```

This is a production-grade cohort analysis query combining all three set operations in a single report.

Student

Can we now go through all 10 practice problems?

ChatGPT • AI Assistant

CONCEPT CARD

Here are 10 problems covering all set operations. Attempt each before reading the solution.

Problem 1 — Easy: UNION — Combine the names and cities of all current and last month's customers into a deduplicated list.

sql – Code Example

```
SELECT customer_id, name, city FROM customers
UNION
SELECT customer_id, name, city FROM last_month_customers
ORDER BY name;
```

Problem 2 — Easy: UNION ALL — Combine all orders from this month and last month into a single list with all rows.

sql – Code Example

```
SELECT order_id, customer_id, order_date, total_amount, status
FROM orders
UNION ALL
SELECT order_id, customer_id, order_date, total_amount, status
FROM last_month_orders
ORDER BY order_date DESC;
```

Problem 3 — Easy: INTERSECT — Find customer IDs who ordered in both this month and last month.

sql – Code Example

```
SELECT customer_id FROM orders
INTERSECT
SELECT customer_id FROM last_month_orders
ORDER BY customer_id;
```

Problem 4 — Easy: EXCEPT — Find customer IDs who ordered last month but NOT this month (churned).

sql – Code Example

```
SELECT customer_id FROM last_month_orders
EXCEPT
SELECT customer_id FROM orders
ORDER BY customer_id;
```

Problem 5 — Medium: UNION ALL with aggregate — Total combined revenue and order count across both months.

sql – Code Example

```
SELECT SUM(total_amount) AS combined_revenue,  
       COUNT(*)          AS combined_orders  
FROM (  
    SELECT total_amount FROM orders  
    UNION ALL  
    SELECT total_amount FROM last_month_orders  
) all_orders;
```

Problem 6 — Medium: INTERSECT with details — Retained customers (ordered both months) with their name and city.

sql – Code Example

```
SELECT c.customer_id, c.name, c.city  
FROM customers c  
WHERE c.customer_id IN (  
    SELECT customer_id FROM orders  
    INTERSECT  
    SELECT customer_id FROM last_month_orders  
)  
ORDER BY c.city, c.name;
```

Problem 7 — Medium: EXCEPT with details — Churned customers with their name, city, and signup date.

sql – Code Example

```
SELECT c.customer_id, c.name, c.city, c.signup_date  
FROM customers c  
WHERE c.customer_id IN (  
    SELECT customer_id FROM last_month_orders  
    EXCEPT  
    SELECT customer_id FROM orders  
)  
ORDER BY c.signup_date;
```

Problem 8 — Medium: EXCEPT rewritten as anti-join (MySQL compatible).

sql – Code Example

```
SELECT DISTINCT c.customer_id, c.name, c.city  
FROM customers c  
JOIN last_month_orders lmo ON c.customer_id = lmo.customer_id  
LEFT JOIN orders o ON c.customer_id = o.customer_id
```

```
WHERE o.customer_id IS NULL  
ORDER BY c.name;
```

Problem 9 — Hard: Full cohort report — Count of new, retained, and churned customers in one query.

sql – Code Example

```
SELECT 'New'          AS cohort, COUNT(*) AS customer_count  
FROM (  
    SELECT customer_id FROM orders  
    EXCEPT  
    SELECT customer_id FROM last_month_orders  
) n  
UNION ALL  
SELECT 'Retained', COUNT(*)  
FROM (  
    SELECT customer_id FROM orders  
    INTERSECT  
    SELECT customer_id FROM last_month_orders  
) r  
UNION ALL  
SELECT 'Churned', COUNT(*)  
FROM (  
    SELECT customer_id FROM last_month_orders  
    EXCEPT  
    SELECT customer_id FROM orders  
) c  
ORDER BY cohort;
```

Problem 10 — Hard: Monthly revenue breakdown using UNION ALL — combine three months (Jan, Feb, Mar 2024) filtered from the single orders table using date ranges, group by month, sort by month.

sql – Code Example

```
SELECT order_month,  
       COUNT(*)          AS order_count,  
       SUM(total_amount) AS monthly_revenue  
FROM (  
    SELECT 'Jan 2024' AS order_month, total_amount  
    FROM orders  
    WHERE order_date ≥ '2024-01-01' AND order_date < '2024-02-01'  
  
    UNION ALL  
  
    SELECT 'Feb 2024' AS order_month, total_amount  
    FROM orders  
    WHERE order_date ≥ '2024-02-01' AND order_date < '2024-03-01'  
  
    UNION ALL  
  
    SELECT 'Mar 2024' AS order_month, total_amount  
    FROM orders  
    WHERE order_date ≥ '2024-03-01' AND order_date < '2024-04-01'  
)  
GROUP BY order_month  
ORDER BY order_month;
```

```
SELECT 'Feb 2024', total_amount
FROM orders
WHERE order_date ≥ '2024-02-01' AND order_date < '2024-03-01'

UNION ALL

SELECT 'Mar 2024', total_amount
FROM orders
WHERE order_date ≥ '2024-03-01' AND order_date < '2024-04-01'
) monthly_data
GROUP BY order_month
ORDER BY order_month;
```

SUMMARY CARD

SQL 1.5 Key Takeaways:

- Set operations combine result sets **vertically** (rows). JOINS combine tables **horizontally** (columns).
- **Hard rule:** both SELECT statements must have the same number of columns with compatible types. Columns matched by position, named by first SELECT.
- **ORDER BY** goes at the very end — applies to the entire combined result.
- **UNION** — deduplicates. Slower. Use only when you need to remove duplicates.
- **UNION ALL** — keeps all rows. Faster. Prefer this whenever duplicates are acceptable or impossible.
- **INTERSECT** — rows in both sets. Equivalent: INNER JOIN + DISTINCT. MySQL (pre-8.0.31) workaround: JOIN rewrite.
- **EXCEPT** — rows in first set, not in second. Order matters. Equivalent: anti-join (LEFT JOIN + WHERE IS NULL).
- **Oracle MINUS** = EXCEPT. Learn both keywords.
- **NULL in set operations:** two NULLs are treated as equal for deduplication — unlike in WHERE clauses.
- Chain set operations left to right. Use parentheses for explicit ordering.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior SQL mentor. I just learned SQL set operations — UNION, UNION ALL, INTERSECT, EXCEPT. Give me 10 progressively harder problems using an e-commerce schema with tables: customers(customerid, name, city, signupdate), orders(orderid, customerid, orderdate, totalamount, status), lastmonthorders (same

structure). Include problems on cohort analysis (new, retained, churned), multi-source data merging, and performance decisions between UNION and UNION ALL. After each problem: wait for my answer, show the correct SQL, explain the set operation choice, show the JOIN equivalent, and give one interview question. Do not continue until I answer.'

🔍 CHECKPOINT QUESTION

Before moving to SQL 1.6 (Constraints and Schema Design): without running it, identify all mistakes in this query and write the corrected version:

sql — Code Example

```
SELECT customer_id, name, city FROM customers ORDER BY name
UNION ALL
SELECT customer_id, city FROM last_month_customers
UNION
SELECT customer_id, name, city, signup_date FROM customers
WHERE city = 'Mumbai';
```

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.5 — set operations, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Set Operations — Vertical Stacking vs JOIN Horizontal Combining:** A two-panel comparison diagram. Left panel (label 'JOIN — Horizontal Combining')...